

MEDS Lab Module 4

Section 6: Combinational Design Practices

Task 3: Half Adder and Full Adder

July 23, 2026

1. EDA Playground Link

The complete design and testbench code for the Half Adder and Full Adder implementations can be found at the following shareable link:

- <https://edaplayground.com/x/W3AL>

2. Schematic Diagram

Below is the top-level block diagram showing the structural composition of the Full Adder using two Half Adders and an OR gate.

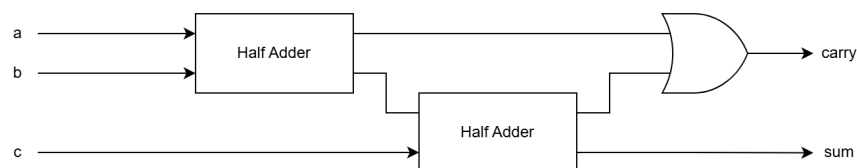


Figure 1: Structural composition of a Full Adder using two Half Adders.

3. Terminal Output Screenshot

The following screenshot displays the simulation log, confirming identical and correct outputs for all input combinations across both implementations.

```

Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 21 02:35 2026
-----Output When a=0 b=0 c_in=0-----
Half Adder Using Assign: sum=0 carry=0
Half Adder Using Always Comb: sum=0 carry=0
Full Adder : sum=0 carry=0
-----Output When a=0 b=1 c_in=0-----
Half Adder Using Assign: sum=1 carry=0
Half Adder Using Always Comb: sum=1 carry=0
Full Adder : sum=1 carry=0
-----Output When a=1 b=0 c_in=1-----
Half Adder Using Assign: sum=1 carry=0
Half Adder Using Always Comb: sum=1 carry=0
Full Adder : sum=0 carry=1
-----Output When a=1 b=1 c_in=1-----
Half Adder Using Assign: sum=0 carry=1
Half Adder Using Always Comb: sum=0 carry=1
Full Adder : sum=1 carry=1
V C S   S i m u l a t i o n   R e p o r t
Time: 40 ns
CPU Time:      0.540 seconds;      Data structure size:  0.0Mb
Tue Jul 21 02:35:10 2026
Done

```

Figure 2: Simulation log output from EDA Playground verifying the Half and Full Adders.

5. Explanation

In this task, a Half Adder was implemented using two different SystemVerilog constructs within the same file: continuous `assign` statements and a procedural `always_comb` block. A Full Adder was then created by structurally instantiating these Half Adders alongside an OR gate.

The `assign` version operates via continuous dataflow modeling, making it ideal for simple combinational expressions. In contrast, the `always_comb` block represents procedural combinational logic, which is often used for multi-line logic or `if-else/case` structures.

Despite these syntactical differences, both versions synthesize to the exact same gate-level hardware. Synthesis tools extract the underlying Boolean truth table represented by the code. Because both constructs define the exact same purely combinational relationship ($Sum = a \oplus b$, $Carry = a \cdot b$) and neither infers sequential memory or latches, the synthesizer optimizes them into the identical minimum hardware equivalent (an XOR gate and an AND gate).